

GIÁO ÁN TRI THỨC

XÂY DỰNG PHÒNG KỸ THUẬT AI ĐA PHIÊN VỚI HERDR

I. THÔNG TIN CHUNG

1. Tên chuyên đề

Thiết kế, vận hành và tối ưu phòng kỹ thuật AI đa phiên bằng Herdr theo mô hình human-like orchestrator.

2. Phạm vi của giáo án

Giáo án trình bày hệ thống tri thức về:

- Bản chất và vai trò thực sự của Herdr.
- Sự khác biệt giữa sub-agent và phiên làm việc độc lập.
- Cách tổ chức một phòng kỹ thuật AI có nhiều agent cùng làm việc.
- Vai trò của root/orchestrator, implementer và peer.
- Cách giao việc để không làm suy giảm năng lực của agent.
- Các sai lầm phổ biến như pre-solve, weak-scout conclusion, balloon pattern và brake pattern.
- Cách chuyển giao ngữ cảnh hiệu quả.
- Quản lý tài nguyên, quyền chỉnh sửa, kiểm thử và evidence.
- Theo dõi metadata của session.
- Continuous optimization và cơ chế monitor.
- Cách triển khai từng bước, tránh black box và tránh kiến trúc quá phức tạp.

Đây là **giáo án tri thức**, không bao gồm bài tập hoặc phần thực hành bắt buộc.

3. Đối tượng phù hợp

- Người đang sử dụng Codex hoặc coding agent để phát triển phần mềm.
- Người muốn điều phối nhiều phiên AI cùng làm việc trên một dự án.
- Technical lead, software architect hoặc project owner muốn xây dựng một “phòng kỹ thuật AI”.
- Người đã có harness, AGENTS.md, CLAUDE.md hoặc hệ thống instruction riêng.
- Người muốn tận dụng nhiều model nhưng không muốn phụ thuộc vào cơ chế sub-agent truyền thống.

4. Mục tiêu tổng quát

Sau khi nắm vững giáo án, người học cần hiểu được rằng:

Herdr không phải một bộ máy tự động quyết định toàn bộ quy trình làm việc. Herdr chủ yếu cung cấp một giao thức nhỏ để các phiên AI có thể được tổ chức và điều phối. Chất lượng thực tế phụ thuộc vào workflow do người sử dụng thiết kế trên giao thức đó.

Mục tiêu cuối cùng là xây dựng được một hệ thống trong đó:

- Có một đầu mối điều hành duy nhất.
- Các agent chuyên môn vẫn giữ được năng lực suy luận độc lập.
- Không biến agent mạnh thành một công cụ chỉ trả lời đúng/sai.
- Không để nhiều agent tranh quyền, sửa cùng một khu vực hoặc chạy cùng một tài nguyên.
- Quy trình luôn minh bạch, có thể quan sát, có thể chỉnh sửa và không phụ thuộc vào “magic”.
- Hệ thống được cải tiến liên tục dựa trên hành vi thực tế.

PHẦN I. TƯ DUY NỀN TẢNG VỀ HERDR

Bài 1. Bản chất của Herdr

1.1. Herdr là một protocol, không phải một workflow hoàn chỉnh

Cách hiểu quan trọng nhất là xem Herdr tương tự như tmux:

- Tmux cung cấp pane, window và session.
- Tmux không ép người dùng phải tổ chức công việc theo một quy trình cố định.
- Người dùng tự quyết định pane nào làm gì, ai điều khiển và cách truyền thông tin.

Tương tự, Herdr cung cấp những khả năng cần thiết để:

- Tạo và quản lý nhiều phiên agent.
- Gửi yêu cầu tới từng phiên.
- Theo dõi trạng thái của các phiên.
- Điều phối các phiên như một phòng ban.
- Gắn metadata hoặc telemetry vào từng phiên.
- Xây dựng workflow đa agent trên một lớp giao tiếp chung.

Herdr không nên tự động quyết định:

- Agent nào phải làm vai trò gì.
- Mọi task phải được chia như thế nào.
- Task nào luôn có độ ưu tiên cao nhất.
- Agent nào được quyền sửa code.
- Khi nào phải mở agent mới.
- Khi nào phải compact hoặc đóng session.
- Mọi dự án đều phải vận hành theo một state machine giống nhau.

Những quyết định trên thuộc về workflow do người sử dụng và root/orchestrator xây dựng.

1.2. Giá trị của một protocol nhỏ

Một protocol nhỏ có các ưu điểm:

- Dễ hiểu.
- Dễ kiểm soát.
- Dễ thay thế từng thành phần.
- Dễ quan sát ảnh hưởng lên Codex và harness.
- Không buộc dự án đi theo một kiến trúc duy nhất.
- Có thể cùng tồn tại với harness hiện có.
- Ít phát sinh ceremony không cần thiết.

Mục tiêu không phải là đưa càng nhiều logic vào Herdr càng tốt. Mục tiêu là giữ Herdr đủ nhỏ để nó trở thành nền móng giao tiếp, còn workflow được phát triển một cách có chủ đích ở phía trên.

1.3. Nguyên tắc “clear is better than clever”

Khi xây hệ thống điều phối AI, cần ưu tiên:

- Instruction rõ hơn cơ chế ngầm.
- File cấu hình rõ hơn suy luận tự động không quan sát được.
- Script đơn giản hơn một framework nhiều lớp.
- Quyền hạn rõ hơn cơ chế thương lượng mơ hồ.
- Log và telemetry rõ hơn phỏng đoán.
- Hook có thể đọc được hơn automation ẩn.

Một hệ thống có vẻ thông minh nhưng người vận hành không hiểu nó đang làm gì sẽ nhanh chóng trở thành black box.

Khi xảy ra lỗi, người sử dụng phải trả lời được:

- Agent nào đã ra quyết định?
- Quyết định dựa trên thông tin nào?
- Agent nào có quyền sửa file?
- Tại sao task này được ưu tiên?
- Tại sao session cũ được tiếp tục thay vì mở session mới?
- Tại sao nhiều agent cùng chạy test?
- Tại sao root tiếp tục polling?
- Vì sao một agent biết hoặc không biết về Herdr?

Nếu không trả lời được, hệ thống đang thiếu tính minh bạch.

Bài 2. Không nhảm Herdr với một framework quản lý agent hoàn chỉnh

2.1. Không cài thêm hệ thống chỉ vì nó có một cơ chế hay

Một công cụ khác có thể chứa một vài mechanism hữu ích, nhưng điều đó không đồng nghĩa phải cài toàn bộ công cụ.

Nguyên tắc được đề xuất là:

1. Đọc và hiểu cơ chế.
2. Xác định cơ chế nào thực sự giải quyết vấn đề.
3. Trích xuất ý tưởng cần thiết.
4. Tự triển khai cơ chế đó trong workflow hiện tại.
5. Không kéo theo các abstraction hoặc dependency không cần thiết.

Trong nội dung nguồn, Firstmate được dùng làm ví dụ:

- Không nên cài Firstmate chỉ vì một vài cơ chế của nó có giá trị.
- Có thể nghiên cứu cách nó xử lý một vấn đề cụ thể.
- Sau đó tích hợp ý tưởng phù hợp vào harness hoặc workflow hiện tại.
- Tuyệt đối tránh biến một dependency thành black box điều khiển toàn bộ hệ thống.

2.2. Không tham vọng xây hệ thống hoàn chỉnh ngay từ đầu

Một sai lầm phổ biến là muốn ngay lập tức xây:

- Nhiều phòng ban.
- Nhiều loại agent.
- Cơ chế negotiation phức tạp.
- State machine hoàn chỉnh.
- Hệ thống priority tự động.
- Context compression đa tầng.
- Memory dài hạn.
- Agent monitor.
- Auto hot reload.
- Nhiều loại proof và evidence.
- Hệ thống tự sửa workflow.

Cách làm tốt hơn:

1. Bắt đầu bằng một root.
2. Thêm một implementer.
3. Thêm một peer khi thực sự cần góc nhìn thứ hai.
4. Quan sát hành vi.
5. Vá một vấn đề cụ thể.
6. Ghi lại nguyên tắc mới.

7. Tiếp tục mở rộng từng bước.

Workflow tốt thường được hình thành từ quá trình sử dụng, quan sát và sửa lỗi, không phải từ một bản thiết kế quá lớn được xây trước khi có dữ liệu thực tế.

PHẦN II. PHÂN BIỆT SUB-AGENT VÀ MULTI-SESSION DEDICATED THREAD

Bài 3. Hạn chế của sub-agent truyền thống

3.1. Tín hiệu quyền lực ảnh hưởng đến hành vi agent

Khi một model được khởi tạo với tư cách sub-agent, system hoặc harness thường nói rõ rằng:

- Nó là sub-agent.
- Nó chỉ sở hữu một bounded subtask.
- Nó phải tuân thủ phạm vi do main agent đặt ra.
- Nó không nên thay đổi hướng.
- Nó không nên chất vấn quyết định cấp trên.
- Nó phải báo cáo ngắn gọn.
- Nó nên tạo một deliverable hoàn chỉnh trong phạm vi nhỏ.
- Nó cần tránh mở rộng vấn đề.

Những instruction này làm agent trở nên ngoan ngoãn và dễ kiểm soát, nhưng đồng thời có thể làm suy giảm khả năng:

- Phát hiện giả định sai của main agent.
- Chất vấn kiến trúc.
- Đề xuất hướng tiếp cận khác.
- Mở rộng không gian giải pháp.
- Nhìn thấy vấn đề nền tảng.
- Từ chối một task được thiết kế sai.
- Tư duy như một kỹ sư ngang hàng.

Một model mạnh vẫn có thể hành xử máy móc nếu nó nhận được tín hiệu rằng nhiệm vụ của nó chỉ là thực thi một phần nhỏ đã được quyết định trước.

3.2. Sub-agent dễ bị biến thành function

Ví dụ, main agent cần xác định kết quả của một vấn đề.

Cách giao sai:

Tôi đã phân tích và kết luận đáp án là A. Hãy kiểm tra xem A có đúng không. Trả lời đúng hoặc sai.

Agent nhận việc gần như chỉ còn là một hàm xác minh.

Nó không có cơ hội:

- Tự xác định vấn đề.
- Tìm các giả thuyết thay thế.
- Kiểm tra tiền đề.
- Xây dựng cách giải riêng.
- Phát hiện rằng câu hỏi ban đầu bị đặt sai.
- Đề xuất rằng một vấn đề nền tảng cần được giải trước.

Kết quả là một agent mạnh bị giảm xuống thành bộ kiểm tra true/false.

3.3. Dedicated thread là một mô hình khác

Trong mô hình Herdr được đề xuất:

- Root mở một phiên làm việc mới.
- Phiên đó là một dedicated thread độc lập.
- Phiên đó không phải sub-agent của Codex.
- Agent trong phiên mới hoạt động như một main agent bình thường.
- Agent có không gian suy luận, khám phá và chất vấn rộng hơn.
- Root giao tiếp với agent thông qua Herdr.
- Agent có thể tưởng rằng yêu cầu đến từ người dùng.

Sự khác biệt cốt lõi không chỉ nằm ở kỹ thuật tạo session. Nó nằm ở **instruction và cảm nhận quyền lực của agent**.

Dedicated thread cho phép agent hành xử gần với một kỹ sư độc lập hơn là một worker bị giới hạn bởi main agent.

3.4. Chỉ nên có một protocol quản lý nhân sự

Nếu cùng lúc sử dụng:

- Sub-agent của Codex.
- Multi-session của Herdr.
- Một framework agent khác.
- Skill tự điều phối.
- Agent con có quyền spawn thêm agent.

Thì hệ thống có nhiều protocol quản lý nhân sự cùng tồn tại.

Hậu quả:

- Không rõ agent nào thuộc quyền ai.
- Không rõ ai được spawn phiên mới.
- Agent con có thể tự mở thêm agent.

- Root không còn là đầu mối duy nhất.
- Có nhiều instruction xung đột.
- Khó theo dõi luồng quyền lực.
- Khó giới hạn tài nguyên.
- Khó xác định trách nhiệm.

Nguyên tắc được đề xuất:

Một phòng kỹ thuật chỉ nên có một giao thức quản lý nhân sự.

Nếu đã chọn Herdr làm protocol điều phối, nên tắt hoặc loại bỏ những rule và skill liên quan đến sub-agent truyền thống, trừ khi có một lý do đặc biệt và một contract rất rõ ràng.

PHẦN III. KIẾN TRÚC PHÒNG KỸ THUẬT AI

Bài 4. Mô hình một root duy nhất

4.1. Root là chủ phòng kỹ thuật

Root/orchestrator là agent duy nhất:

- Biết đầy đủ về Herdr.
- Biết cấu trúc phòng ban.
- Có quyền mở và điều phối các phiên.
- Chọn agent nào xử lý task nào.
- Theo dõi trạng thái từng session.
- Phân phối ngữ cảnh.
- Quản lý quyền chỉnh sửa.
- Quản lý quyền chạy test và evidence.
- Reconcile độ ưu tiên.
- Phát hiện agent bị kẹt.
- Quyết định tiếp tục session cũ hay mở session mới.
- Tổng hợp và phản biện kết quả.
- Báo cáo cuối cùng cho người dùng.

Root không nhất thiết phải là model mạnh nhất trong mọi trường hợp.

Với công việc thông thường, một model medium có khả năng điều phối tốt có thể đủ cho vai trò root. Giá trị chính của root nằm ở:

- Khả năng tổ chức.
- Khả năng đặt câu hỏi.
- Khả năng nhận biết giới hạn.
- Khả năng đối chiếu nhiều quan điểm.
- Khả năng phân bổ tài nguyên.

- Khả năng nhận biết foundation yếu.
- Khả năng tổng hợp quyết định.

4.2. Root phải human-like

Một root human-like không phải là agent tự giải mọi việc rồi phân nhỏ phần dư cho người khác.

Root human-like hành xử như technical lead:

1. Hiểu yêu cầu và mục tiêu.
2. Xác định phần chưa rõ.
3. Chọn đúng người để tham khảo.
4. Đặt câu hỏi mở.
5. Để người được hỏi tự nghiên cứu.
6. Đọc kỹ ý kiến.
7. Challenge và phản biện.
8. So sánh các hướng.
9. Ra quyết định.
10. Giao quyền thực thi rõ ràng.
11. Theo dõi nhưng không micromanage.
12. Kiểm tra evidence trước khi kết luận.

Root không được xem các agent khác là những model kém hơn mặc định.

Trong nhiều trường hợp, peer hoặc implementer có năng lực tương đương root, chỉ khác vai trò và quyền hạn.

4.3. Root không nên tự động pre-solve

Pre-solve là hành vi root:

- Tự đọc phần lớn code.
- Tự hình thành kết luận.
- Tự chọn giải pháp.
- Sau đó mới gọi agent khác để xác nhận.

Đây là một lỗi điều phối vì:

- Root đã đóng khung không gian giải pháp.
- Agent khác chỉ nhìn vấn đề qua giả định của root.
- Nhiều hướng có giá trị bị loại trước khi được khám phá.
- Token của agent khác bị dùng để xác nhận thay vì tạo tri thức mới.
- Root dễ mắc confirmation bias.

Cách đúng là giao một câu hỏi mở trước khi root kết luận.

Ví dụ:

Hãy phân tích khu vực xác thực này từ đầu. Xác định các vấn đề kiến trúc, rủi ro và những thay đổi có tác động lớn nhất. Không giả định rằng hướng hiện tại là đúng.

Sau khi nhận kết quả, root mới:

- Kiểm tra bằng chứng.
- Đặt câu hỏi phản biện.
- So sánh với ý kiến khác.
- Chọn hướng cuối cùng.

Bài 5. Ba profile cốt lõi

Đối với phần lớn công việc thông thường, phòng kỹ thuật chỉ cần ba loại profile chính.

5.1. Root/orchestrator

Đặc điểm:

- Có toàn bộ instruction về Herdr.
- Có quyền quản lý department.
- Có quyền mở và đóng session.
- Có quyền gửi task.
- Có quyền quản lý lock.
- Có quyền điều phối evidence.
- Có quyền reconcile priority.
- Không nhất thiết trực tiếp sửa code.
- Không nên dùng skill Herdr có thể bị mất sau compact.
- Instruction cốt lõi nên nằm trực tiếp trong profile.

Lý do không nên đặt instruction điều phối quan trọng trong skill:

- Skill có thể cần được load lại.
- Sau compact, agent có thể không còn giữ đầy đủ nội dung skill.
- Quyền điều phối là hành vi nền tảng, không phải kiến thức tùy chọn.
- Root phải luôn nhớ cấu trúc phòng ban và giới hạn quyền lực.

5.2. Owner/implementer

Owner/implementer là người sở hữu một feature hoặc một phạm vi chỉnh sửa.

Đặc điểm:

- Là profile chính có quyền edit.
- Nhận một mục tiêu rõ ràng.
- Có quyền thực hiện thay đổi trong phạm vi được giao.
- Không được biết về Herdr nếu không thật sự cần.

- Không được biết mình được một agent khác invoke.
- Nên cảm nhận yêu cầu như đến trực tiếp từ user.
- Không có quyền điều phối các agent khác.
- Không có instruction về cách quản lý department.
- Không được tự ý mở rộng quyền sửa sang khu vực khác.

Một feature chỉ nên có một owner có quyền edit tại một thời điểm.

Điều này giúp tránh:

- Hai agent sửa cùng file.
- Xung đột git.
- Chồng chéo implementation.
- Một agent vô tình phá thiết kế của agent khác.
- Không rõ ai chịu trách nhiệm cuối cùng.
- Evidence không khớp với code đang thay đổi.

5.3. Peer

Peer được root tạo ra tùy mục đích:

- Hỏi ý kiến kiến trúc.
- Phản biện thiết kế.
- Review code.
- Phân tích rủi ro.
- Kiểm tra giả định.
- Tìm vấn đề nền tảng.
- So sánh hai phương án.
- Đọc evidence.
- Đánh giá kế hoạch.

Peer thường không có quyền edit, trừ khi root chuyển giao ownership rõ ràng.

Peer cần giữ khả năng suy luận độc lập. Vì vậy, root không nên đưa sẵn kết luận và yêu cầu peer xác nhận.

Bài 6. Các profile chuyên biệt tùy chọn

Ngoài ba profile cốt lõi, có thể tạo các profile chuyên biệt để giảm lượng prompt lặp lại.

Ví dụ:

- `root.config.toml`
- `peer.config.toml`
- `implementer.config.toml`
- `reviewer.config.toml`
- `scout.config.toml`

- `proof-auditor.config.toml`
- `shadow-co-worker.config.toml`

Mục đích của profile chuyên biệt là nạp sẵn các giới hạn ổn định.

6.1. Reviewer

- Chỉ đọc.
- Không chỉnh sửa code.
- Tập trung vào correctness, maintainability và risk.
- Không tự biến review thành refactor.
- Trích dẫn file, method hoặc evidence cụ thể.
- Phân biệt lỗi chắc chắn với đề xuất cải tiến.

6.2. Scout

- Khảo sát codebase.
- Tìm file, module, method và luồng thực thi liên quan.
- Tạo artifact dẫn đường.
- Không kết luận một vấn đề kiến trúc khó nếu năng lực model không đủ.
- Không được quyết định giải pháp cuối cùng.
- Không sửa code.

6.3. Proof auditor

- Đọc test, log, trace và evidence.
- Xác định evidence có thực sự chứng minh claim hay không.
- Phát hiện test giả, test flaky hoặc test không bao phủ yêu cầu.
- Không mặc định “test xanh” đồng nghĩa tính năng đúng.

6.4. Shadow co-worker

- Quan sát cách root và implementer làm việc.
- Tìm điểm lãng phí.
- Đề xuất cải tiến workflow.
- Không can thiệp trực tiếp nếu chưa được trao quyền.
- Có thể làm dữ liệu đầu vào cho continuous optimization.

6.5. Không nên tạo quá nhiều profile sớm

Mỗi profile mới làm tăng:

- Số lượng cấu hình cần duy trì.
- Khả năng instruction xung đột.
- Chi phí chọn agent.
- Độ phức tạp quan sát.
- Khó khăn khi xác định quyền hạn.

Chỉ tạo profile mới khi một hành vi lặp đi lặp lại và việc viết lại instruction mỗi lần gây tốn kém rõ ràng.

PHẦN IV. NGUYÊN TẮC GIAO VIỆC

Bài 7. Đặt câu hỏi mở để mở rộng không gian nhận thức

7.1. Câu hỏi đóng làm giảm năng lực agent

Những câu hỏi sau thường có chất lượng thấp:

- “Giải pháp này đúng không?”
- “Có phải lỗi nằm ở method X không?”
- “Chọn phương án A hay B.”
- “Hãy xác nhận kiến trúc này ổn.”
- “Tôi nghĩ cần thêm mutex, hãy triển khai.”
- “Tôi đã xác định root cause là cache, kiểm tra giúp.”

Các câu hỏi này:

- Gắn agent vào giả định của root.
- Giới hạn số phương án.
- Khuyến khích agent chiều theo cấp trên.
- Tạo confirmation bias.
- Biến agent thành công cụ xác nhận.

7.2. Câu hỏi mở phù hợp hơn

Câu hỏi mở nên yêu cầu agent tự xây dựng mô hình vấn đề.

Ví dụ:

- “Hãy phân tích nguyên nhân có thể gây ra lỗi này và xếp hạng bằng chứng.”
- “Hãy đánh giá kiến trúc hiện tại từ nguyên lý đầu tiên.”
- “Điều gì trong foundation có thể khiến feature này trở nên khó duy trì?”
- “Nếu không bị ràng buộc bởi giải pháp hiện tại, anh/chị sẽ thiết kế phần này như thế nào?”
- “Hãy tìm các thay đổi có leverage lớn nhất.”
- “Những giả định nào trong kế hoạch này chưa được chứng minh?”
- “Hãy tìm một phương án có thể khiến kế hoạch hiện tại không còn cần thiết.”

7.3. Quy trình hỏi – nghe – challenge

Một quy trình điều phối tốt gồm ba giai đoạn.

Giai đoạn 1: Hỏi mở

Root cung cấp:

- Mục tiêu.
- Bối cảnh cần thiết.
- Phạm vi.
- Những constraint thật sự.
- Yêu cầu về evidence.

Root không cung cấp sẵn đáp án.

Giai đoạn 2: Lắng nghe

Root đọc kết quả với thái độ:

- Tìm insight mới.
- Tìm giả định root chưa thấy.
- Tìm mâu thuẫn.
- Tìm vấn đề foundation.
- Tìm phương án có leverage tốt hơn.

Giai đoạn 3: Challenge

Root đặt câu hỏi phản biện:

- Bằng chứng nào hỗ trợ kết luận?
- Điều gì có thể làm kết luận sai?
- Phương án này tạo debt nào?
- Có giải pháp nền tảng hơn không?
- Có thay đổi P2 nào giúp loại bỏ hoàn toàn P0 hiện tại không?
- Nếu không dùng abstraction hiện tại thì sao?
- Evidence nào cần có trước khi edit?

Challenge không phải bác bỏ agent. Mục tiêu là nâng chất lượng reasoning trước khi quyết định.

Bài 8. Sử dụng model yếu đúng cách

8.1. Sai lầm: cho model yếu scout rồi kết luận

Model yếu có thể nhanh và rẻ khi:

- Tìm file.
- Liệt kê symbol.
- Tạo bản đồ thư mục.
- Tìm nơi một method được gọi.
- Trích xuất prompt.

- Phân loại dữ liệu.
- Tạo artifact sơ bộ.

Nhưng không nên giao cho model yếu kết luận chắc chắn về:

- Root cause phức tạp.
- Kiến trúc hệ thống.
- Tính đúng đắn của một refactor lớn.
- Bảo mật.
- Concurrency.
- Tính toàn vẹn dữ liệu.
- Lựa chọn foundation.
- Một quyết định có blast radius lớn.

Nếu model yếu đưa ra kết luận sai, root phải:

- Đọc lại code.
- Kiểm tra lại toàn bộ.
- Sửa mental model.
- Tốn nhiều token hơn so với tự phân tích từ đầu.

8.2. Vai trò đúng của scout model yếu

Scout chỉ nên tạo artifact dẫn đường, chẳng hạn:

- Danh sách file liên quan.
- Call graph sơ bộ.
- Bản đồ module.
- Những method có khả năng có impact cao.
- Vị trí test.
- Danh sách log hoặc trace cần đọc.
- Các vùng code chưa hiểu.
- Các giả thuyết cần model mạnh xác minh.

Một báo cáo scout tốt có dạng:

Tôi tìm thấy method A được gọi từ B, C và D. Thay đổi tại A có thể ảnh hưởng tới ba luồng này.
 Tôi chưa kết luận A là root cause; đây là vùng có leverage cao nên được phân tích tiếp.

Đây là cách model yếu dẫn đường mà không vượt quá năng lực.

8.3. Phân chia theo năng lực nhận thức

Có thể dùng:

- Model yếu cho extraction.
- Model yếu cho indexing và navigation.
- Model medium cho orchestration thông thường.

- Model medium cho implementation có phạm vi rõ.
- Model mạnh cho kiến trúc, phản biện sâu hoặc foundation.
- Nhiều peer độc lập cho vấn đề có rủi ro cao.

Không nên chỉ dựa vào nhãn “main” hoặc “sub”. Chất lượng phụ thuộc vào:

- Instruction.
- Không gian tự chủ.
- Bối cảnh.
- Quyền lực cảm nhận.
- Cách đặt câu hỏi.
- Loại task.

PHẦN V. FOUNDATION VÀ CÁC ANTI-PATTERN

Bài 9. Không xây feature đẹp trên nền móng sai

9.1. Model mạnh vẫn có thể làm đẹp một kiến trúc sai

Một model mạnh thường có khả năng:

- Thêm abstraction.
- Thêm lock.
- Thêm cache.
- Thêm retry.
- Thêm mutex.
- Thêm queue.
- Thêm adapter.
- Thêm heuristic.
- Làm test xanh.
- Tạo giao diện hoặc feature rất thuyết phục.

Điều nguy hiểm là model có thể sử dụng năng lực này để **bù trừ cho một foundation sai**, thay vì chất vấn foundation.

Kết quả là hệ thống:

- Có vẻ hoạt động.
- Có nhiều kỹ thuật phức tạp.
- Có feature mới.
- Có test.
- Nhưng càng phát triển càng khó duy trì.

9.2. Balloon pattern

Balloon pattern là hình ảnh ẩn dụ:

Móng nhà yếu nhưng thay vì sửa móng, kỹ sư gắn khình khí cầu để nâng ngôi nhà lên.

Trong phần mềm, balloon pattern xuất hiện khi:

- Kiến trúc nền tảng không phù hợp.
- Feature mới cần quá nhiều workaround để tồn tại.
- Thêm abstraction để che một mô hình sai.
- Thêm concurrency primitives để cứu thiết kế không cần concurrency.
- Thêm state machine để quản lý một luồng vốn có thể đơn giản.
- Thêm caching để bù cho data flow sai.
- Thêm retry để che lỗi lifecycle.
- Thêm synchronization để vá ownership không rõ.

Balloon pattern nguy hiểm vì mỗi workaround có thể hợp lý khi nhìn riêng lẻ, nhưng toàn bộ hệ thống vẫn dựa trên tiền đề sai.

9.3. Brake pattern

Brake pattern dùng hình ảnh một chiếc xe chưa có phanh nhưng vẫn tiếp tục được nâng cấp.

Ví dụ:

- Hệ thống chưa có tính toàn vẹn dữ liệu nhưng tiếp tục thêm tính năng.
- API chưa có authorization đúng nhưng tiếp tục mở rộng endpoint.
- Kiến trúc networking sai nhưng tiếp tục xây gameplay.
- Không có ownership rõ nhưng tiếp tục tăng số agent.
- Không có evidence đáng tin nhưng tiếp tục tự động deploy.
- Không có lock tài nguyên nhưng tăng số worker chạy test.

Trước khi tăng tốc, phải bảo đảm hệ thống có “phanh”:

- Boundary.
- Validation.
- Ownership.
- Rollback.
- Evidence.
- Observability.
- Permission.
- Failure handling.

9.4. Ví dụ về lựa chọn foundation

Nguồn đưa ra một ví dụ về một hệ thống game nhiều người dùng:

- Ban đầu chọn kiến trúc async và sử dụng nhiều cơ chế Tokio.
- Sau đó nhận ra mô hình đúng có thể nên là sync hoặc sans-I/O.
- Tuy nhiên, các model mạnh khi được giao feature mới vẫn tiếp tục xây dựng trên nền async.
- Chúng thêm lock, `Arc`, `Mutex` và heuristic để làm feature hoạt động.

- Feature có thể hào nhoáng, nhưng nền tảng vẫn không phù hợp.

Bài học:

Agent không tự động biết rằng foundation phải bị thay thế. Nếu task chỉ yêu cầu xây feature, agent mạnh có thể tối ưu rất giỏi trong một không gian giải pháp sai.

Do đó root phải luôn hỏi:

- Foundation có đúng không?
- Constraint hiện tại là constraint thật hay chỉ là di sản?
- Feature này có đang tạo thêm balloon không?
- Có nên dừng implementation để sửa nền trước không?

Bài 10. Xây dựng tài liệu anti-pattern chung

10.1. Vai trò của `ANTI_PATTERN.md`

Một tài liệu như `ANTI_PATTERN.md` giúp human và agent có một ngôn ngữ chung.

Nó có thể định nghĩa:

- Balloon pattern.
- Brake pattern.
- Pre-solve pattern.
- Weak-scout conclusion.
- Polling waste.
- Dual ownership.
- Evidence collision.
- Black-box workflow.
- Over-compression.
- Priority-by-label.
- Agent hierarchy collapse.
- Frozen-wait mismatch.
- Feature-over-foundation.

Mỗi anti-pattern nên có:

1. Tên.
2. Mô tả.
3. Dấu hiệu nhận biết.
4. Nguyên nhân.
5. Hậu quả.
6. Cách xử lý.
7. Ví dụ trong codebase.

10.2. Giá trị của việc đặt tên

Khi một pattern đã có tên, root có thể giao tiếp ngắn gọn:

- “Kiểm tra xem kế hoạch này có balloon pattern không.”
- “Đừng để scout đưa ra weak-scout conclusion.”
- “Task này đang bị priority-by-label.”
- “Hai agent đang rơi vào dual ownership.”
- “Root đang mắc polling waste.”
- “Implementation này đang feature-over-foundation.”

Tên gọi tạo ra một từ vựng chung giúp giảm prompt và tăng độ nhất quán.

10.3. Đặt tên cho kế hoạch lớn

Ngoài anti-pattern, một kế hoạch lớn cũng nên có tên riêng.

Ví dụ:

- Plan Big Bang.
- Foundation Reset.
- Design System Renewal.
- Auth Boundary Repair.
- Netcode Simplification.

Khi kế hoạch có tên:

- Dễ reference trong session khác.
- Dễ báo cáo tiến độ.
- Dễ liên kết issue.
- Dễ phân biệt task cục bộ và chương trình dài hạn.
- Dễ lưu vào memory.
- Dễ yêu cầu agent đọc đúng bối cảnh.

PHẦN VI. HỆ THỐNG HARNESS MINH BẠCH

Bài 11. Herdr phải cùng tồn tại với harness

11.1. Herdr không thay thế harness

Harness vẫn chịu trách nhiệm cho các thành phần như:

- Instruction dự án.
- Quy tắc coding.
- Tooling.

- Script.
- Test command.
- Evidence generation.
- Hook.
- Planning support.
- Validation.
- Repository conventions.
- AGENTS.md.
- CLAUDE.md.
- Skill cần thiết.
- Cách agent đọc và sửa code.

Herdr thêm lớp điều phối nhiều session, không thay thế các quy tắc phát triển phần mềm hiện có.

11.2. Kiểm tra các instruction hiện có

Trước khi thêm workflow Herdr, cần xác định:

- Dự án đã có `AGENTS.md` chưa?
- Dự án đã có `CLAUDE.md` chưa?
- Có rule sub-agent nào không?
- Có skill nào được phép spawn hoặc điều phối agent không?
- Có rule xung đột quyền edit không?
- Có instruction về test và evidence không?
- Có hook chạy sau mỗi turn không?
- Có script tạo plan không?
- Có quy trình issue hoặc task hiện tại không?

Nếu đã dùng Herdr làm protocol duy nhất, nên xem xét loại bỏ hoặc vô hiệu hóa:

- Skill sub-agent không còn dùng.
- Rule yêu cầu main agent luôn spawn sub-agent.
- Instruction cho phép mọi agent tự điều phối.
- Prompt khiến implementer biết mình chỉ là agent con.
- Cơ chế tự động xung đột với quyền của root.

11.3. Hạng chế ceremony

Mỗi thành phần thêm vào hệ thống phải trả lời được:

- Nó giải quyết vấn đề thực tế nào?
- Vấn đề đã xảy ra bao nhiêu lần?
- Có giải pháp đơn giản hơn không?
- Nó ảnh hưởng tới prompt caching thế nào?
- Nó làm tăng context bao nhiêu?
- Người dùng có quan sát được không?
- Có thể tắt riêng không?
- Có cơ chế rollback không?

Những thành phần không chứng minh được giá trị nên được xem là ceremony.

11.4. Không biến workflow thành black box

Một bộ setup chia sẻ sẵn có thể giúp bắt đầu nhanh, nhưng cũng có nguy cơ:

- Người dùng không biết bên trong có gì.
- Không biết instruction nào đang tác động agent.
- Không biết workflow có hợp dự án không.
- Không biết cách sửa khi hệ thống lỗi.
- Phụ thuộc vào tác giả.
- Khó nâng cao kỹ năng điều phối.

Tự setup từng bước có lợi vì người dùng:

- Hiểu từng rule.
- Biết tại sao rule tồn tại.
- Biết chỗ cần sửa.
- Xây workflow phù hợp với thói quen thật.
- Nắm rõ tác động lên harness.

PHẦN VII. CHUYỂN GIAO NGỮ CẢNH

Bài 12. Không fork toàn bộ lịch sử một cách mù quáng

12.1. Chi phí của “fork turn all”

Khi coordinator cần giao việc cho agent khác, việc gửi toàn bộ chat history có thể:

- Tốn rất nhiều token.
- Chứa nhiều thông tin không liên quan.
- Mang theo các giả định cũ.
- Làm agent mới mất thời gian lọc.
- Làm nguội prompt cache.
- Lặp lại log và trace dài.
- Gây nhiễu mental model.

Thay vì fork toàn bộ, root nên tạo context pack có chọn lọc.

12.2. Một context pack tốt

Context pack nên gồm:

1. Mục tiêu.
2. Trạng thái hiện tại.

3. Những gì đã được xác minh.
4. Những gì chưa rõ.
5. Constraint thực sự.
6. File hoặc module liên quan.
7. Quyết định đã chốt.
8. Quyết định còn mở.
9. Evidence hiện có.
10. Deliverable mong muốn.
11. Quyền được phép.
12. Những anti-pattern cần tránh.

Context pack không nên chứa mọi chi tiết lịch sử nếu các chi tiết đó không ảnh hưởng đến quyết định hiện tại.

Bài 13. Sử dụng hình ảnh để đóng gói thông tin

13.1. Những dữ liệu phù hợp với biểu diễn hình ảnh

Hình ảnh có thể có ROI cao với:

- Project structure.
- Monorepo structure.
- Quan hệ giữa module.
- Dependency graph.
- Call graph.
- Data flow.
- Architecture map.
- Sequence diagram.
- State relationship.
- Chart.
- Graph.
- Console output dài.
- Log trace.
- Bản đồ lịch sử dự án.
- Long-term memory dạng quan hệ.
- Tóm tắt nhiều session.

Một hình ảnh tốt có thể giúp agent nhận biết cấu trúc tổng thể nhanh hơn một khối text dài.

13.2. Không chuyển mọi thứ thành hình

Quan điểm hoàn thiện hơn trong nguồn là:

Chỉ chuyển sang hình những dữ liệu chấp nhận được tính lossy. Không nên chuyển toàn bộ harness hoặc instruction cốt lõi thành hình.

Không nên phụ thuộc hoàn toàn vào hình đối với:

- System instruction.
- Instruction bắt buộc.
- AGENTS .md .
- CLAUDE .md .
- Rule bảo mật.
- Quy định quyền hạn.
- Code chính xác.
- Command cần chạy chính xác.
- Contract API.
- Tiêu chí acceptance bắt buộc.
- Thông tin cần prompt caching ổn định.

Lý do:

- Hình ảnh có thể làm mất chi tiết.
- Agent có thể đọc sai text nhỏ.
- Khó diff.
- Khó version control.
- Không tận dụng prompt caching của text.
- Không phù hợp với nội dung yêu cầu tính chính xác tuyệt đối.

13.3. Nguyên tắc kết hợp text và image

Cách dùng cân bằng:

- Text giữ rule, instruction và fact chính xác.
- Image giữ quan hệ, topology và dữ liệu dài để nhìn.
- Context pack có bản tóm tắt text ngắn.
- Hình ảnh đóng vai trò bản đồ.
- Agent có thể quay lại file gốc khi cần chi tiết.
- Không nén quá mức đến mức không thể kiểm chứng.

13.4. Đánh giá hiệu quả bằng thực nghiệm

Có thể đánh giá text pack và image pack theo cách:

1. Tạo một artifact hình ảnh mô tả kiến trúc.
2. Tạo một README text mô tả cùng kiến trúc.
3. Chọn nhiều model tương đương.
4. Một nhóm chỉ đọc text.
5. Một nhóm chỉ đọc hình.
6. Đặt cùng một nhóm câu hỏi.
7. So sánh:
8. Độ chính xác.

9. Khả năng tìm module.
10. Khả năng hiểu quan hệ.
11. Số token sử dụng.
12. Thời gian suy luận.
13. Số lần cần hỏi lại.
14. Dựa trên kết quả để điều chỉnh mức cô đặc.

Không nên mặc định hình luôn tốt hơn text hoặc ngược lại. Hiệu quả phụ thuộc vào loại thông tin.

PHẦN VIII. QUẢN LÝ TASK VÀ ĐỘ ƯU TIÊN

Bài 14. Không sắp xếp công việc chỉ theo P0, P1, P2

14.1. Hạn chế của priority tĩnh

Một issue P0 có thể khẩn cấp, nhưng không phải lúc nào cũng nên làm đầu tiên.

Ví dụ:

- Issue X là P0 và có thể vá ngay.
- Issue Y là P2 nhưng tạo một foundation đúng.
- Nếu làm Y trước, X có thể được giải quyết trọn vẹn hơn.
- Nếu vá X trước, code có thể phải sửa lại sau Y.

Do đó thứ tự hợp lý có thể là:

1. Y.
2. Sau đó X.

Priority không chỉ là mức độ khẩn cấp. Nó còn phụ thuộc:

- Dependency.
- Foundation.
- Shape của giải pháp.
- Chi phí làm lại.
- Khả năng hấp thụ issue.
- Rủi ro dài hạn.
- Blast radius.
- Khả năng mở khóa nhiều task.

14.2. Issue absorption

Một trường hợp đặc biệt:

- Issue X đang tồn tại.
- Một issue hoặc plan Y lớn hơn sẽ loại bỏ hoàn toàn nguyên nhân của X.

- Nếu Y đã được chấp thuận và chắc chắn được triển khai, X có thể không cần sửa riêng.
- X có thể được đóng vì đã được hấp thụ vào Y.

Tuy nhiên, cần bảo đảm:

- Y thực sự bao phủ acceptance của X.
- Không có khoảng thời gian rủi ro quá dài.
- Người liên quan hiểu lý do đóng.
- Tracking giữa X và Y rõ ràng.
- Evidence sau Y xác minh X đã biến mất.

14.3. Reconcile task định kỳ

Sau mỗi ba hoặc bốn task, root nên đánh giá lại:

- Task nào còn cần thiết?
- Task nào bị thay đổi bởi implementation mới?
- Priority nào đã lỗi thời?
- Có foundation task nào nên đưa lên trước?
- Issue nào đã được hấp thụ?
- Có task nào nên chia nhỏ?
- Có task nào không còn đúng giả định?
- Agent nào đang giữ ownership?
- Tài nguyên nào đang bị khóa?
- Plan lớn có thay đổi không?

Reconcile không phải chỉ là sort lại bảng. Đây là quá trình reasoning về hình dạng hệ thống.

14.4. Có thể dùng nhiều peer để reconcile

Đối với kế hoạch quan trọng, root có thể hỏi nhiều peer độc lập:

- Peer A ưu tiên theo rủi ro.
- Peer B ưu tiên theo foundation.
- Peer C ưu tiên theo giá trị người dùng.
- Peer D tìm issue có thể hấp thụ issue khác.

Root sau đó tổng hợp và challenge các quan điểm trước khi quyết định.

PHẦN IX. QUẢN LÝ QUYỀN, TEST VÀ EVIDENCE

Bài 15. Quyền chỉnh sửa phải rõ ràng

15.1. Một owner cho một phạm vi

Tại một thời điểm:

- Một feature có một owner.
- Một file hoặc vùng code nhạy cảm không nên có nhiều implementer cùng sửa.
- Peer và reviewer mặc định chỉ đọc.
- Chuyển giao ownership phải được root ghi nhận.
- Agent cũ phải dừng sửa trước khi agent mới bắt đầu.

15.2. Tách quyền suy luận và quyền chỉnh sửa

Một agent có thể:

- Được phép phân tích toàn bộ hệ thống.
- Nhưng chỉ được sửa một module.
- Hoặc không được sửa gì.

Điều này giúp tận dụng khả năng nhận thức rộng mà không gây conflict.

Bài 16. Lock cho test và evidence nặng

16.1. Vì sao cần lock

Trong dự án có test hoặc evidence nặng, nhiều agent chạy đồng thời có thể:

- Dẫn lên database test.
- Tranh port.
- Làm CPU hoặc RAM quá tải.
- Ghi đè artifact.
- Làm log trộn lẫn.
- Khiến test timeout.
- Tạo false red.
- Tạo false green do dùng cache hoặc artifact cũ.
- Làm môi trường flaky hơn.

Do đó phải có lock.

16.2. Root là người trao quyền chạy

Root nên quản lý:

- Ai được chạy full test.
- Ai được chạy integration test.
- Ai được dùng database test.
- Ai được chạy benchmark.
- Ai được tạo proof artifact.
- Ai được dùng GPU.
- Ai được sử dụng port hoặc service cụ thể.

Các agent khác có thể chuẩn bị command nhưng không chạy nếu chưa có lock.

16.3. Trạng thái lock cần minh bạch

Mỗi lock nên có:

- Tên tài nguyên.
- Agent đang sở hữu.
- Thời điểm cấp.
- Task liên quan.
- Điều kiện giải phóng.
- Timeout.
- Artifact đầu ra dự kiến.

Root phải xử lý lock bị bỏ quên khi agent:

- Crash.
- Freeze.
- Đóng session.
- Trả về `done` nhưng không release.
- Bị compact mất trạng thái.

16.4. Đề phòng flaky environment

Khi test đỏ, không được kết luận code sai ngay.

Cần phân biệt:

- Lỗi code thật.
- Lỗi môi trường.
- Race do nhiều agent.
- Port conflict.
- Dữ liệu test bị ô nhiễm.
- Artifact cũ.
- Cache cũ.
- Timeout do tải máy.

- Test vốn đã flaky.

Evidence phải kèm bối cảnh môi trường để proof auditor đánh giá.

PHẦN X. METADATA VÀ KINH TẾ SESSION

Bài 17. Metadata cần thiết cho mỗi session

Mỗi session nên hiển thị metadata hữu ích như:

- Compact count.
- Phần trăm context window còn lại.
- Số token đã sử dụng.
- Cached token.
- Tỷ lệ token được cache.
- Thời gian idle.
- Cache đang hot hay cold.
- Trạng thái hiện tại.
- Task đang sở hữu.
- Quyền edit.
- Lock đang giữ.
- Lần cập nhật gần nhất.

Metadata giúp root đưa ra quyết định giống một người điều phối có kinh nghiệm.

17.1. Context còn lại

Nếu session chỉ còn ít context:

- Một turn mới có thể đẩy session vào compact.
- Agent có thể mất chi tiết.
- Một câu hỏi lớn có thể không còn hiệu quả.
- Có thể nên tạo session mới và gửi context pack.

Tuy nhiên, không phải cứ context thấp là phải đóng ngay. Root còn phải xem:

- Cache có hot không?
- Agent đang giữ mental model quý giá không?
- Câu hỏi tiếp theo lớn hay nhỏ?
- Compact đã xảy ra bao nhiêu lần?
- Context pack có đủ tốt để chuyển giao không?

17.2. Cache hot và cache cold

Nếu cache còn hot:

- Gửi thêm một turn có thể có chi phí thấp.
- Agent vẫn giữ ngữ cảnh hiệu quả.
- Tiếp tục session có thể tốt hơn tạo mới.

Nếu cache đã cold:

- Turn mới có thể cần xử lý lại lượng token rất lớn.
- Một session mới với context pack cô đọng có thể rẻ hơn.
- Root cần cân nhắc giá trị của mental model đang tồn tại.

17.3. Không biến metadata thành state machine cứng

Metadata nên cung cấp thông tin, không nên tự động ép agent theo một matrix cứng.

Ví dụ không nên có rule máy móc:

- Context dưới 20% luôn mở session mới.
- Idle hơn 10 phút luôn đóng.
- Compact hai lần luôn dừng.
- Cache cold luôn bỏ session.
- Task P0 luôn dùng model mạnh nhất.

Thay vào đó, root nhận metadata và suy luận theo tình huống.

Mục đích là tăng nhận thức, không thay thế judgment.

Bài 18. Cập nhật telemetry bằng hook

Nguồn đề xuất cập nhật metadata tại các thời điểm:

- SessionStart
- PostCompact
- Stop

Ví dụ cấu hình:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
```

```

        "command": "bash '/root/.codex/herdr-agent-state.sh' session",
        "timeout": 10,
        "type": "command"
    },
    {
        "command": "python3 '/root/.codex/herdr-pane-telemetry.py'",
        "timeout": 10,
        "type": "command"
    }
]
},
],
"PostCompact": [
    {
        "hooks": [
            {
                "command": "python3 '/root/.codex/herdr-pane-telemetry.py'",
                "timeout": 10,
                "type": "command"
            }
        ]
    }
],
"Stop": [
    {
        "hooks": [
            {
                "command": "python3 '/root/.codex/herdr-pane-telemetry.py'",
                "timeout": 10,
                "type": "command"
            }
        ]
    }
]
}
}
}

```

Ý nghĩa:

- **SessionStart** : khởi tạo trạng thái session và thu telemetry ban đầu.
- **PostCompact** : cập nhật dữ liệu sau khi context bị compact.
- **Stop** : ghi nhận trạng thái cuối hoặc trạng thái tạm dừng.
- Script telemetry có thể xuất metadata để Herdr hiển thị trên pane.

Các script cần:

- Chạy nhanh.

- Không gây block session.
- Có timeout.
- Ghi dữ liệu theo cách atomic.
- Không làm hỏng workflow nếu telemetry lỗi.
- Có log riêng.
- Không chứa logic điều phối quá lớn.

PHẦN XI. MONITOR VÀ CONTINUOUS OPTIMIZATION

Bài 19. Vì sao cần một monitor bên ngoài

Một phòng kỹ thuật đa agent thường phát sinh hành vi không dự đoán trước:

- Root chờ sai trạng thái.
- Agent trả `done` nhưng root chờ `idle`.
- Root polling liên tục.
- Agent giữ lock sau khi hoàn thành.
- Nhiều agent cùng chạy test.
- Agent con biết về Herdr do instruction rò rỉ.
- Root giao câu hỏi đóng.
- Implementer tự mở rộng ownership.
- Session cold vẫn bị tiếp tục dùng.
- Context bị compact nhiều lần.
- Cùng một lỗi workflow lặp lại.

Một monitor bên ngoài có thể:

- Quan sát các phiên.
- Đọc telemetry.
- Phát hiện pattern kém hiệu quả.
- Đề xuất thay đổi instruction.
- Cập nhật script hoặc config.
- Hot reload thay đổi.
- Tiếp tục quan sát sau thay đổi.

19.1. Monitor không phải một root thứ hai

Monitor không nên:

- Giao task nghiệp vụ.
- Tranh quyền điều phối.
- Tự thay đổi priority.
- Ra lệnh trực tiếp cho implementer.

- Tự quyết định kiến trúc sản phẩm.

Monitor tập trung vào **hiệu quả của quy trình**, không điều hành nội dung dự án.

19.2. Chu trình continuous optimization

Chu trình chuẩn:

1. Quan sát.
2. Thu thập trace.
3. Phát hiện hành vi lãng phí.
4. Xác định nguyên nhân.
5. Đề xuất một thay đổi nhỏ.
6. Hot reload.
7. Tiếp tục quan sát.
8. So sánh trước và sau.
9. Giữ hoặc rollback thay đổi.
10. Ghi lại tri thức vào workflow.

Continuous optimization là quá trình không có điểm kết thúc tuyệt đối.

Workflow cần phát triển theo:

- Loại dự án.
- Model đang sử dụng.
- Khối lượng task.
- Giá token.
- Giới hạn context.
- Chất lượng prompt caching.
- Tooling.
- Các lỗi từng xảy ra.

Bài 20. Chống polling waste

20.1. Vấn đề

Root có thể liên tục hỏi:

- Agent xong chưa?
- Agent đang làm gì?
- Trạng thái đã đổi chưa?
- Có kết quả chưa?

Mỗi lần polling có thể:

- Tốn context của root.

- Tốn token.
- Làm loãng mental model.
- Làm cache kém hiệu quả.
- Gây nhiễu lịch sử.
- Không tạo giá trị nếu trạng thái chưa đổi.

20.2. Hướng cải tiến

Ưu tiên:

- Event-driven notification.
- Trạng thái được cập nhật qua hook.
- Agent chủ động báo khi hoàn thành.
- Root chỉ kiểm tra khi có tín hiệu.
- Backoff nếu buộc phải polling.
- Tách trạng thái `done`, `idle`, `blocked` và `waiting`.
- Có timeout hợp lý.
- Monitor phát hiện polling bất thường.

20.3. Phân biệt `done` và `idle`

Một lỗi có thể xảy ra:

- Root chờ trạng thái `idle`.
- Co-worker hoàn thành và chuyển sang `done`.
- Root không coi `done` là điều kiện kết thúc.
- Root tiếp tục chờ hoặc freeze.

Cần định nghĩa rõ:

- `working`: đang thực hiện.
- `blocked`: cần thông tin hoặc tài nguyên.
- `done`: đã hoàn thành deliverable.
- `idle`: chưa có task.
- `stopped`: session dừng.
- `error`: không thể tiếp tục.

Logic điều phối phải hiểu rằng `done` có thể là tín hiệu root cần thu kết quả, không phải tiếp tục chờ `idle`.

PHẦN XII. CÁCH XÂY WORKFLOW TỪ LỊCH SỬ LÀM VIỆC

Bài 21. Khai thác các session cũ

Đối với người không muốn tự mô tả workflow từ đầu, có thể khai thác lịch sử sử dụng Codex.

Quy trình được đề xuất:

Giai đoạn 1: Trích xuất prompt người dùng

Dùng một model có chi phí thấp để đọc khoảng 100 session gần nhất và trích xuất:

- Prompt của người dùng.
- Loại task.
- Cách người dùng sửa yêu cầu.
- Các constraint lặp lại.
- Những điều người dùng thường không hài lòng.
- Cách người dùng yêu cầu xác minh.
- Cách người dùng chia giai đoạn.
- Cách người dùng đặt tên file và artifact.

Model ở giai đoạn này chỉ extraction, không diễn giải sâu.

Giai đoạn 2: Trích xuất workflow pattern

Dùng một model khác để tìm pattern:

- Người dùng thường bắt đầu task như thế nào?
- Khi nào người dùng yêu cầu đọc code toàn bộ?
- Người dùng ưu tiên độ chính xác hay tốc độ?
- Người dùng thường yêu cầu plan trước hay làm ngay?
- Người dùng thích artifact nào?
- Người dùng xử lý khi agent sai như thế nào?
- Có quy trình review lặp lại không?
- Người dùng thường xác nhận qua các giai đoạn nào?
- Những instruction nào thường xuyên được nhắc lại?

Kết quả là một mô hình workflow của người dùng.

Giai đoạn 3: Mô phỏng workflow bằng model mạnh

Dùng model mạnh để:

- Đánh giá các pattern.

- Loại bỏ pattern ngẫu nhiên.
- Phân biệt preference và constraint.
- Mô phỏng cách người dùng điều hành.
- Chuyển workflow thành instruction cho root.
- Xác định bước nào nên dùng script.
- Xác định bước nào cần judgment.
- Đề xuất cấu trúc profile.

Giai đoạn 4: Kiểm chứng và tinh chỉnh

Không sử dụng kết quả như chân lý tuyệt đối.

Cần:

- Bắt đầu với phạm vi nhỏ.
- Quan sát root.
- So sánh với cách người dùng thật sự làm.
- Sửa instruction khi root mô phỏng sai.
- Không tự động đưa toàn bộ pattern vào hệ thống.
- Loại bỏ thói quen cũ không còn phù hợp.

PHẦN XIII. QUY TRÌNH VẬN HÀNH CHUẨN

Bài 22. Vòng đời của một task

Bước 1. Root tiếp nhận yêu cầu

Root xác định:

- Mục tiêu.
- Deliverable.
- Constraint.
- Mức độ rủi ro.
- Phạm vi code.
- Evidence cần có.
- Foundation có đáng nghi không.

Bước 2. Kiểm tra tri thức hiện có

Root đọc:

- AGENTS.md.
- CLAUDE.md.
- Tài liệu kiến trúc.
- ANTI_PATTERN.md.

- Plan liên quan.
- Memory.
- Task và issue hiện tại.
- Telemetry session.

Bước 3. Quyết định có cần fan-out không

Không phải task nào cũng cần nhiều agent.

Không fan-out khi:

- Task nhỏ.
- Phạm vi rõ.
- Một implementer có thể hoàn thành.
- Chi phí chuyển giao lớn hơn lợi ích.
- Không có quyết định kiến trúc.

Fan-out khi:

- Vấn đề có nhiều giả thuyết.
- Kiến trúc chưa rõ.
- Rủi ro cao.
- Cần scout.
- Cần review độc lập.
- Cần proof auditor.
- Cần so sánh phương án.

Bước 4. Chọn profile

- Root giữ orchestration.
- Scout nếu cần dẫn đường.
- Peer nếu cần phản biện.
- Implementer khi đã xác định ownership.
- Reviewer sau implementation.
- Proof auditor khi evidence phức tạp.

Bước 5. Tạo context pack

Context pack phải vừa đủ:

- Không gửi toàn bộ lịch sử.
- Không giấu constraint.
- Không đưa sẵn đáp án.
- Có đường dẫn tới artifact gốc.
- Có quyền và giới hạn rõ ràng.
- Có định nghĩa hoàn thành.

Bước 6. Giao câu hỏi mở

Root yêu cầu agent:

- Phân tích độc lập.
- Trình bày bằng chứng.
- Nêu giả định.
- Chỉ ra rủi ro.
- Phân biệt fact và inference.
- Không vượt quá quyền chỉnh sửa.

Bước 7. Thu kết quả và challenge

Root:

- Không chấp nhận kết luận chỉ vì agent mạnh.
- Kiểm tra evidence.
- So sánh peer.
- Tìm mâu thuẫn.
- Hỏi về foundation.
- Kiểm tra anti-pattern.
- Quyết định hướng.

Bước 8. Trao ownership

Root chọn một implementer và xác định:

- Phạm vi sửa.
- File hoặc module.
- Acceptance criteria.
- Test được phép chạy.
- Lock được cấp.
- Artifact cần tạo.
- Điều kiện báo blocked.

Bước 9. Theo dõi bằng trạng thái, không micromanage

Root dựa trên:

- Event.
- Telemetry.
- Trạng thái session.
- Báo cáo blocked.
- Kết quả hook.

Không polling liên tục nếu không cần.

Bước 10. Review và proof

Sau implementation:

- Reviewer đọc thay đổi.
- Proof auditor kiểm tra evidence.
- Root đánh giá lỗi foundation.
- Chạy test theo lock.
- Phân biệt lỗi code và lỗi môi trường.

Bước 11. Reconcile

Root cập nhật:

- Task.
- Priority.
- Issue absorption.
- Plan.
- Memory.
- Anti-pattern.
- Ownership.
- Lock.
- Session cần giữ hoặc đóng.

Bước 12. Kết thúc hoặc tiếp tục

Root quyết định:

- Tiếp tục session hiện tại.
 - Mở session mới.
 - Compact.
 - Lưu context pack.
 - Đóng implementer.
 - Giữ peer cho giai đoạn tiếp theo.
 - Báo cáo kết quả cho người dùng.
-

PHẦN XIV. CÁC LỖI THƯỜNG GẶP VÀ CÁCH KHẮC PHỤC

Bài 23. Bảng lỗi vận hành

Lỗi 1: Mọi agent đều biết Herdr

Biểu hiện

- Agent con nói về pane, root hoặc department.
- Agent con tự mở agent khác.
- Agent con cố điều phối workflow.

Nguyên nhân

Instruction Herdr được đưa vào mọi profile.

Khắc phục

- Chỉ root có full Herdr instruction.
 - Implementer và peer không biết protocol nếu không cần.
 - Xóa skill hoặc rule Herdr khỏi agent con.
-

Lỗi 2: Root tự giải rồi yêu cầu xác nhận

Biểu hiện

- Task gửi cho peer chỉ có lựa chọn A/B.
- Peer trả lời ngắn, không có insight mới.
- Root luôn nhận được kết luận giống mình.

Nguyên nhân

Pre-solve và confirmation bias.

Khắc phục

- Giao câu hỏi trước khi root chốt mental model.
 - Yêu cầu agent phân tích từ đầu.
 - Tách exploration và decision.
-

Lỗi 3: Model yếu đưa ra kết luận khó

Biểu hiện

- Scout tuyên bố root cause chắc chắn.
- Root phải đọc lại toàn bộ code.
- Kết quả scout gây lệch hướng.

Nguyên nhân

Không giới hạn vai trò scout.

Khắc phục

- Scout chỉ tạo artifact dẫn đường.
 - Yêu cầu ghi mức độ chắc chắn.
 - Kết luận khó phải được peer mạnh hoặc root xác minh.
-

Lỗi 4: Nhiều implementer cùng sửa

Biểu hiện

- Conflict.
- Test không ổn định.
- Code thay đổi qua lại.
- Không rõ ai chịu trách nhiệm.

Nguyên nhân

Không có ownership và lock.

Khắc phục

- Một owner cho mỗi phạm vi.
 - Reviewer mặc định read-only.
 - Chuyển giao ownership rõ ràng.
-

Lỗi 5: Root polling liên tục

Biểu hiện

- Context root đầy bởi câu hỏi trạng thái.
- Token tăng nhưng không có insight.
- Agent chưa xong vẫn bị hỏi.

Nguyên nhân

Thiếu event hoặc telemetry.

Khắc phục

- Dừng hook.
 - Tạo trạng thái rõ.
 - Backoff.
 - Monitor polling frequency.
-

Lỗi 6: Root chờ `idle`, agent trả `done`

Biểu hiện

- Workflow freeze.
- Agent đã hoàn thành nhưng root không thu kết quả.

Nguyên nhân

State semantics không thống nhất.

Khắc phục

- Định nghĩa trạng thái.
 - Xem `done` là tín hiệu cần xử lý.
 - Không xây state machine quá phức tạp nhưng phải thống nhất ý nghĩa.
-

Lỗi 7: Xây feature trên foundation sai

Biểu hiện

- Nhiều lock, retry, wrapper và heuristic.
- Feature hoạt động nhưng code ngày càng phức tạp.
- Mỗi thay đổi nhỏ cần nhiều workaround.

Nguyên nhân

Balloon pattern.

Khắc phục

- Dừng feature.
- Đánh giá foundation.

- Tạo plan sửa nền.
 - Reconcile priority.
-

Lỗi 8: Sắp task thuần theo nhãn

Biểu hiện

- Vá P0 rồi phải làm lại sau P2.
- Foundation task luôn bị trì hoãn.
- Nhiều fix cục bộ.

Nguyên nhân

Priority-by-label.

Khắc phục

- Đánh giá dependency và leverage.
 - Xem xét issue absorption.
 - Reconcile sau mỗi vài task.
-

Lỗi 9: Chuyển toàn bộ harness thành hình ảnh

Biểu hiện

- Agent bỏ sót rule.
- Prompt caching kém.
- Không thể diff instruction.
- Khó kiểm chứng text.

Nguyên nhân

Over-compression.

Khắc phục

- Giữ instruction cốt lõi ở dạng text.
 - Chỉ dùng hình cho dữ liệu quan hệ và dữ liệu chấp nhận lossy.
 - Luôn giữ nguồn gốc để tra cứu.
-

Lỗi 10: Cài quá nhiều framework

Biểu hiện

- Không biết ai điều phối.
- Nhiều protocol.
- Rule xung đột.
- Hệ thống trở thành black box.

Nguyên nhân

Tích hợp toàn bộ công cụ chỉ vì một mechanism hay.

Khắc phục

- Chọn một protocol.
- Trích xuất mechanism cần thiết.
- Ưu tiên script và config minh bạch.
- Loại bỏ dependency không tạo ROI.

PHẦN XV. LỘ TRÌNH TRIỂN KHAI

Bài 24. Giai đoạn 1 – Tối thiểu khả dụng

Chỉ triển khai:

- Một root.
- Một implementer.
- Một peer read-only.
- Một protocol Herdr.
- Quyền edit rõ.
- Trạng thái cơ bản.
- Context pack dạng text.
- Không dùng sub-agent Codex song song.

Mục tiêu:

- Xác minh root có giao việc đúng không.
- Xác minh implementer giữ được năng lực main-agent.
- Xác minh quyền hạn không xung đột.

Bài 25. Giai đoạn 2 – Chuẩn hóa workflow

Bổ sung:

- Profile config riêng.
- ANTI_PATTERN.md .
- Quy trình ownership.
- Lock test.
- Definition of done.
- Reconcile định kỳ.
- Reviewer hoặc proof auditor khi cần.

Mục tiêu:

- Giảm prompt lặp.
 - Tăng tính nhất quán.
 - Giảm lỗi vận hành.
-

Bài 26. Giai đoạn 3 – Telemetry

Bổ sung:

- Context left.
- Compact count.
- Token cache.
- Idle time.
- Hot/cold status.
- Hook SessionStart .
- Hook PostCompact .
- Hook Stop .

Mục tiêu:

- Root đưa quyết định session tốt hơn.
 - Giảm chi phí token.
 - Tránh tiếp tục session cold không cần thiết.
-

Bài 27. Giai đoạn 4 – Continuous optimization

Bổ sung:

- Monitor bên ngoài.
- Phát hiện polling.
- Phát hiện frozen state.

- Hot reload config.
- Log thay đổi workflow.
- Đo hiệu quả trước và sau.

Mục tiêu:

- Quy trình tự thích nghi với hành vi thực tế.
 - Không để lỗi vận hành trở thành thói quen.
-

Bài 28. Giai đoạn 5 – Mở rộng đa dự án

Chỉ mở rộng khi workflow một dự án đã ổn định.

Có thể bổ sung:

- Nhiều department.
- Root cấp cao điều phối nhiều root dự án.
- Shared monitor.
- Shared telemetry.
- Resource lock liên dự án.
- Memory và architecture map dùng chung.

Tuy nhiên phải giữ nguyên tắc:

- Mỗi department có một đầu mối điều hành rõ.
 - Không để root cấp cao micromanage implementer.
 - Không trộn ownership giữa các dự án.
 - Không biến hệ thống thành ma trận quyền lực không thể quan sát.
-

PHẦN XVI. HỆ THỐNG NGUYÊN TẮC CỐT LÕI

Bài 29. Hai mươi nguyên tắc cần ghi nhớ

Nguyên tắc 1

Herdr là protocol, không phải workflow hoàn chỉnh.

Nguyên tắc 2

Workflow do người sử dụng xây trên protocol.

Nguyên tắc 3

Chỉ có một root được phép điều hành department.

Nguyên tắc 4

Dedicated thread không phải sub-agent.

Nguyên tắc 5

Không sử dụng đồng thời nhiều protocol quản lý nhân sự nếu không có contract cực kỳ rõ.

Nguyên tắc 6

Agent con không cần biết về Herdr.

Nguyên tắc 7

Implementer nên cảm nhận yêu cầu như đến trực tiếp từ user.

Nguyên tắc 8

Không biến agent mạnh thành hàm xác nhận đúng/sai.

Nguyên tắc 9

Root phải hỏi mở, lắng nghe rồi mới challenge.

Nguyên tắc 10

Model yếu được dùng để dẫn đường, không được kết luận vấn đề khó.

Nguyên tắc 11

Một phạm vi chỉ có một owner được quyền edit tại một thời điểm.

Nguyên tắc 12

Test và evidence nặng phải có lock.

Nguyên tắc 13

Không xây feature hào nhoáng trên foundation sai.

Nguyên tắc 14

Phải nhận biết balloon pattern và brake pattern.

Nguyên tắc 15

Priority phải dựa trên dependency và leverage, không chỉ dựa trên P0/P1/P2.

Nguyên tắc 16

Sau mỗi vài task phải reconcile lại kế hoạch.

Nguyên tắc 17

Metadata nên hỗ trợ judgment, không thay thế judgment bằng state machine cứng.

Nguyên tắc 18

Chỉ chuyển sang hình những dữ liệu chấp nhận lossy; instruction cốt lõi vẫn nên ở dạng text.

Nguyên tắc 19

Monitor tập trung tối ưu quy trình, không trở thành root thứ hai.

Nguyên tắc 20

Luôn ưu tiên minh bạch: clear is better than clever.

KẾT LUẬN TOÀN CHUYÊN ĐỀ

Một hệ thống Herdr hiệu quả không được đánh giá bằng số lượng agent, số lượng profile hoặc mức độ tự động hóa.

Nó được đánh giá bằng việc:

- Root có ra quyết định tốt hơn không?
- Các agent có giữ được năng lực suy luận độc lập không?
- Có giảm lãng phí token và context không?
- Có ngăn được xung đột chỉnh sửa không?
- Có phát hiện foundation sai trước khi xây thêm feature không?
- Evidence có đáng tin không?
- Workflow có minh bạch và sửa được không?
- Người sử dụng có hiểu toàn bộ hệ thống không?
- Quy trình có ngày càng tốt hơn qua quan sát thực tế không?

Mô hình tối giản được khuyến nghị cho công việc thông thường là:

1. Root/orchestrator

Điều phối toàn bộ phòng kỹ thuật, giữ instruction Herdr, quản lý quyền, session, priority và evidence.

2. Owner/implementer

Sở hữu một feature hoặc phạm vi chỉnh sửa, là agent duy nhất có quyền edit trong phạm vi đó và không cần biết về Herdr.

3. Peer

Cung cấp góc nhìn độc lập, phản biện, review hoặc phân tích mà không mặc định tuân theo kết luận của root.

Herdr phát huy giá trị cao nhất khi nó vẫn là một protocol nhỏ, còn trí tuệ của hệ thống nằm trong cách root tổ chức con người, phiên làm việc, ngữ cảnh, quyền hạn và quá trình phản biện.

Điều cần xây dựng không phải là một đàn sub-agent thực thi mệnh lệnh máy móc, mà là một phòng kỹ thuật gồm nhiều kỹ sư AI độc lập, được điều hành bởi một technical lead có khả năng suy nghĩ và giao tiếp giống con người.